TileCDC: Content-Defined Chunking on CPU Matrix Units

Ashutosh Rath
github.com/ashutosh-rath02

October 2026

Abstract

Content-defined chunking (CDC) splits data at positions chosen by a rolling hash so that deduplication survives insertions and deletions. Hash-based chunkers such as Gear and FastCDC evaluate the hash one byte at a time through a loop-carried dependency, which caps them near 2 GB/s per core. A windowed hash that is linear in its input, evaluated at every byte offset, is a product of a Toeplitz key matrix and the Hankel matrix of the byte stream. We show that a matrix unit can form the Hankel operand at no cost: an Intel AMX tile load whose row stride (16 bytes) is shorter than its rows (64 bytes) yields overlapping windows without copying, so one `TDPBUSD` instruction computes 256 window hashes. TileCDC combines this kernel with a byte substitution, a multiplicative mix and FastCDC-style normalized chunking. On one vCPU of a 2.1 GHz Xeon it chunks real source and binary datasets at a median 4.5–4.8 GB/s, 2.2–2.6× the faster of two FastCDC implementations (1.7–2.1 GB/s). At equal mean chunk size its deduplication ratio is within -0.9% to +0.3% of FastCDC's (-0.21% on average). The same formulation on AVX-512 reaches 2.4–2.7 GB/s on CPUs without AMX, and all implementations return bit-identical boundaries. TileCDC supports average chunk sizes up to 32 KiB and is open source as a C library and a Python package.

1 Introduction

Deduplicating storage, backup tools and file synchronisation split data into chunks and store each distinct chunk once [2, 4]. Fixed-size chunks fail after a single inserted byte, because every later boundary shifts. Content-defined chunking (CDC) instead places a boundary wherever a hash of the last few dozen bytes satisfies a condition, so boundaries move with the content and most chunks survive an edit [1, 2].

The cost of CDC is the hash. Rabin fingerprints [1] and the lighter Gear hash [3] used by FastCDC [4, 5] are rolling hashes: the value at byte i is computed from the value at byte $i-1$. That dependency chain limits a scalar chunker to roughly one byte every one or two cycles; FastCDC reaches 1.7–2.1 GB/s on our machine. Faster chunkers either drop the hash altogether (hashless chunkers that look for extreme or monotone byte patterns [6, 7, 8]), skip regions predicted to be duplicates [10], or run several sequential hash streams in parallel lanes [11, 8].

This paper takes a different route. Instead of rolling a hash forward, we compute an independent windowed hash at *every* byte offset. If the hash is linear in the window's bytes, it is a one-dimensional convolution of the byte stream with a key, and the hashes of all windows are a matrix product. Server CPUs now contain matrix units built for such products: Intel's Advanced Matrix Extensions (AMX) multiply 16×64 tiles of 8-bit integers in one instruction. The obstacle is that the windows overlap, and writing every window out as a matrix row (im2col) would cost more than the hash. Overlapping strided views are familiar from array libraries and direct-convolution kernels; our observation is that AMX's own tile load accepts a row stride shorter than the row, so with a 16-byte stride and 64-byte rows, row r of the loaded tile is bytes $16r$ to $16r+63$ of the stream. The tile *is* the Hankel matrix of the input, formed by the load unit. Multiplying it by a constant 64×16 Toeplitz tile built from the key yields the hashes of 256 consecutive offsets in one `TDPBUSD` (Figure 1). We found no prior use of a matrix unit for chunking or byte-stream fingerprinting.

Contributions.

- A formulation of windowed byte hashing at every offset as a Hankel-times-Toeplitz product, with the Hankel operand formed by an overlapping-stride tile load (§3).
- TileCDC, a chunker built on it: a byte substitution and a multiplicative mix turn the linear hash into a usable boundary test, and FastCDC-style normalized chunking with early exit keeps deduplication within 1% of FastCDC's.
- An AVX-512 formulation of the same function (`vpermb` sliding windows feeding `vpdpbusd`) for CPUs without AMX, and a portable scalar reference, all producing bit-identical boundaries, checked against an independent oracle in 18,078 comparisons.
- An evaluation on 1 GB of real versioned source and binary data against two FastCDC implementations (§5), and an account of the approach's limits, including a cap on the average chunk size that follows from the hash's entropy (§6).

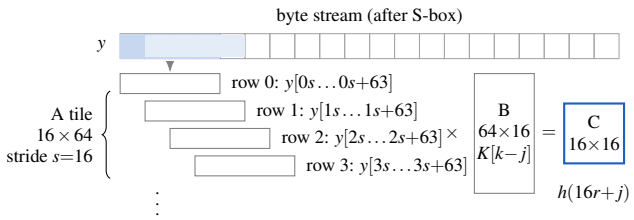


Figure 1: The Hankel-tile kernel. A tile load with row stride 16 over 64-byte rows makes row r the window starting at byte $16r$, with no copy. Multiplying by a constant Toeplitz tile with $B[k][j] = K[k-j]$ gives $C[r][j] = \sum_d K[d]\, y[16r+j+d]$: the window hash at offset $16r+j$. One `TDPBUSD` yields 256 consecutive hashes.

2 Background

CDC and normalized chunking. A chunker scans the input and declares a boundary after byte $c-1$ when a hash of the bytes just before c has its masked bits all zero. With b masked bits a boundary occurs on average every 2^b bytes. FastCDC [4, 5] adds three refinements that TileCDC keeps: it skips hashing for the first *min* bytes of each chunk, cuts unconditionally at *max*, and uses *normalized chunking*: a stricter mask (more bits) before the target average size and a looser one after, which narrows the chunk-size distribution. Its hash is Gear, $h_i = (h_{i-1} \ll 1) + G[x_i]$ with a table G of 256 random 64-bit values.

Intel AMX. AMX adds eight tile registers of up to 16 rows by 64 bytes and a tile matrix-multiply unit [15]. `TILELOADD` loads a tile from memory given a base address and a row stride; `TDPBUSD` multiplies a tile of unsigned bytes (16×64) by a tile of signed bytes (viewed as 64×16) and accumulates into a 16×16 tile of 32-bit integers: 16,384 multiply-accumulates per instruction. AMX is present on Intel Xeon processors since Sapphire Rapids and is available to unprivileged Linux processes after an `arch_prctl` request.

3 Design

3.1 A hash that is a matrix product

Let x be the input and $y_j = S[x_j]$ its bytes after a fixed random permutation S (§3.3). For a key $K \in \{-127, \ldots, 127\}^{48}$ with no zero entries we define the window hash

$$h(i) \;=\; \sum_{d=0}^{47} K[d]\, y_{i+d}, \tag{1}$$

computed exactly in 32-bit integers ($|h| < 2^{21}$). A boundary may follow the window, at chunk end $c = i+48$. Stacking the windows starting at offsets $i_0, \ldots, i_0+255$ as the rows of a matrix Y with $Y[i][d] = y_{i_0+i+d}$ gives a Hankel matrix (constant along anti-diagonals), and the vector of hashes is YK.

3.2 Hankel tiles from the load unit

AMX multiplies 16×64 by 64×16 tiles, so we split the 256 offsets into 16 rows of 16. For row r and column j we need offset $16r+j$. Load the A tile from address $y + i_0$ with row stride 16 and 64-byte rows: row r holds $y_{i_0+16r}, \ldots, y_{i_0+16r+63}$. Let the B tile hold the Toeplitz matrix $B[k][j] = K[k-j]$ for $0 \le k-j < 48$ and zero otherwise. Then

$$\begin{aligned} C[r][j] &= \sum_{k=0}^{63} y_{i_0+16r+k}\, K[k-j] \\ &= \sum_{d=0}^{47} K[d]\, y_{i_0+16r+j+d} \;=\; h(i_0+16r+j). \end{aligned} \tag{2}$$

The 64-byte rows cover the 48-byte window for all 16 column shifts ($j+47 \le 62$). Because the load unit reads overlapping rows directly, no window is ever materialised; each input byte is read by four consecutive rows. The B tile is constant. TileCDC issues four such products per block of 1,024 offsets to overlap their latencies.

3.3 From a linear hash to a boundary test

A linear hash of raw bytes is a poor boundary test: text occupies a narrow byte range, so h takes few distinct values and the boundary rate depends on the data. Two cheap steps fix this. Bytes first pass through a random permutation S, applied 64 bytes at a time with two `vpermi2b` and a blend (AVX-512 VBMI), at 0.026 ns per byte. The hash is then mixed by one multiplication, $m = h \cdot \texttt{0x9E3779B1} \bmod 2^{32}$, and a boundary is allowed when the top b bits of m are zero. The key K and permutation S are derived from a 64-bit seed.

These steps spread the hash but cannot add entropy: h is a sum of 48 bounded terms, roughly Gaussian with about 17 bits of entropy. Masks of 16 or more bits therefore become coarse, and the mean chunk size starts to depend on the seed. On 1 GiB of random data and six seeds, the mean chunk size ranged over 8.1–8.4 KiB for an 8 KiB target, 32.0–37.0 KiB for 32 KiB, and 55.0–74.3 KiB for 64 KiB. TileCDC 0.1 therefore rejects average sizes above 32 KiB (§6).

3.4 Normalized chunking with early exit

TileCDC uses FastCDC's structure with the hash replaced: no hashing in the first *min* bytes of a chunk, a strict mask up to the average size, a loose mask after it, and an unconditional cut at *max*. For a target average of 2^n bytes and normalization level ν (default 1, §5) the masks have $n+\nu$ and $n-\nu-1$ bits; the loose mask has one bit fewer than FastCDC's at the same level. Each block of 1,024 hashes produces two 1,024-bit candidate bitmaps (strict and loose); the scan returns the first strict candidate up to the average size, else the first loose candidate up to the maximum, else the maximum, and stops computing as soon as a cut is found. Work past a cut is at most one block.

3.5 The same function without AMX

The every-offset formulation also maps onto AVX-512. For 16 consecutive offsets, `vpermb` (VBMI) gathers, for each 32-bit lane k, the four bytes $y_{i+4g+k}, \ldots, y_{i+4g+k+3}$ from one 64-byte register, and `vpdpbusd` (VNNI) multiplies them by the four key bytes $K[4g..4g+3]$ and accumulates. Twelve permute/dot-product pairs give 16 hashes, about 1.5 instructions per byte with no dependency between offsets. A scalar loop implements the definition directly. All three implementations consume the same substituted block and produce identical bitmaps, so a chunk boundary never depends on the CPU, a requirement for formats that must deduplicate across machines.

4 Implementation

The C library is about 300 lines with no dependencies beyond libc and pthreads. Each implementation is compiled with a function-level target attribute, so one binary runs everywhere and selects AMX, then AVX-512, then scalar at run time; CPU detection runs once per process. Around each call the library saves the calling thread's tile configuration, installs its own and restores the previous one afterwards; tile register contents are not preserved, so callers must not hold live tile data across a call. Blocks near the end of a buffer are copied into a zero-padded scratch area, so TileCDC never reads past the caller's buffer, and hashes whose windows extend past the end are never consulted. The API is FastCDC's: `tilecdc_cut(data, len)` returns the length of the first chunk, which supports streaming with a buffer of *max* bytes. A CPython extension releases the GIL during chunking and adds file iteration.

5 Evaluation

Setup. All measurements use a virtual machine with two vCPUs of an Intel Xeon at 2.1 GHz nominal (AVX-512 VBMI/VNNI and AMX), 7 GB of memory and Linux 6.18, with GCC 13 at `-O3`. **SRC** is the uncompressed source releases of Django 5.0/5.1/5.2 and NumPy 2.1/2.2/2.3 (6 files, 421 MB). **BIN** is the unpacked x86-64 wheels of NumPy 2.1/2.2, SciPy 1.14/1.15/1.16 and pandas 2.2.0/2.2.3/2.3.0 (8 files, 632 MB), mostly compiled shared libraries. **RAND** is 256 MB of random bytes. Our primary baseline is the 2020 implementation in the `fastcdc` Rust crate (version 3.2, normalization level 1), built with `target-cpu=native`. We also report a C implementation of FastCDC's algorithm that hashes one byte per step with level-2 normalization and the paper's 8 KiB masks [5]; it omits the 2020 paper's two-bytes-per-step optimization and is compiled with `-march=native`. Unless stated, chunk sizes are 2/8/64 KiB (min/avg/max). Single-thread runs are pinned to one vCPU. Each throughput figure is the median of 5 independent runs, each the best of five in-memory passes (sweeps: best of three); ranges across runs reached 34% on this shared VM. Throughput counts all input bytes, including the first *min* bytes of each chunk that no method hashes, and excludes chunk fingerprinting.

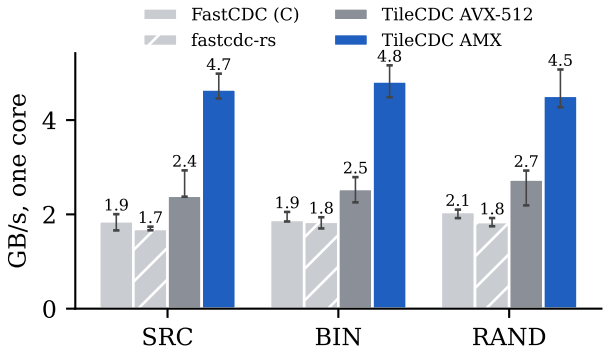


Figure 2: Single-core chunking throughput at 2/8/64 KiB: median of 5 runs, with bars showing the range.

stage (cached block)	ns/byte	GB/s
byte substitution (VBMI)	0.026	39
hash + mix + test, AMX	0.188	5.3
tile loads + 4 `TDPBUSD`	0.061	
storing result tiles	0.017	
multiply and mask tests	0.110	
hash + mix + test, AVX-512	0.407	2.5
hash + mix + test, scalar	1.74	0.58

Table 1: Cost per hashed byte on a cache-resident block.

Throughput. Figure 2 shows single-core throughput. TileCDC on AMX chunks SRC at a median 4.65 GB/s (range 4.46–4.99), BIN at 4.82 and RAND at 4.52 GB/s. The Rust crate reaches 1.69, 1.84 and 1.84 GB/s and the C baseline 1.85, 1.88 and 2.05 GB/s, so TileCDC is 2.2–2.6× faster than the better baseline on each dataset. The AVX-512 path reaches 2.40, 2.54 and 2.74 GB/s, 1.29–1.35× the better baseline, which matters because AMD's current server CPUs have AVX-512 but no AMX. The portable scalar path, a reference for correctness, runs at 0.56 GB/s. Across target averages from 4 to 32 KiB, AMX throughput stays between 4.4 and 5.3 GB/s.

Where the time goes. Table 1 times each stage on a block that fits in L1. The AMX hash, mix and test cost 0.188 ns per hashed byte, about 0.39 cycles at the nominal 2.1 GHz, against 0.86 for AVX-512. The tile multiply is not the bottleneck: timed in parts, the tile loads and four `TDPBUSD` take 0.061 ns per byte (measured with one result tile stored to keep the products live), storing the remaining tiles 0.017, and the vector multiply and mask tests 0.110 (58% of the path), so a cheaper boundary test is the main lever left. With substitution the kernel costs 0.214 ns per hashed byte (4.7 GB/s). End-to-end throughput can match or exceed this figure because the first *min* bytes of each chunk are never hashed but

	method	mean	sd	p5	p95	dedup
SRC	FastCDC (C)	10.54	5.51	3.71	18.46	1.311
	TileCDC	9.56	5.24	2.89	17.09	1.320
BIN	FastCDC (C)	9.67	4.87	3.58	15.04	1.265
	TileCDC	8.79	4.83	2.81	14.21	1.269

Table 2: Chunk sizes (KiB) and deduplication ratio at 2/8/64 KiB. The Rust crate gives means of 11.74 and 10.54 KiB with ratios 1.281 and 1.254.

are counted.

Deduplication. Deduplication ratios depend strongly on the mean chunk size, and the three chunkers produce different means for the same parameters, so we compare them along the curve of ratio against mean chunk size (Figure 3). Interpolating each FastCDC curve (log-linearly) at TileCDC's mean sizes between 8.8 and 37.2 KiB, TileCDC's ratio differs by -0.9% to +0.3%, -0.21% on average; it is lower in 8 of 12 comparisons. TileCDC's smallest target (4 KiB) produces means below the range of the FastCDC curves and is not compared. The normalization level matters: with $\nu = 2$, TileCDC's chunk sizes were more tightly clustered and it lost up to -2.1% against FastCDC; $\nu = 1$ gave the results above, so we made it the default. This level was chosen on the same two datasets used for evaluation; we had no held-out data. Table 2 gives chunk-size statistics at the default parameters.

Resilience to edits. We applied 10, 100 and 1,000 random insertions or deletions of 1–100 bytes to a NumPy source tarball and a SciPy binary tarball, five seeds each, and measured the share of the edited file's bytes that fall in chunks already present before the edit (Table 3). Because a smaller mean chunk size by itself preserves more data, we also ran TileCDC with its average size raised until its mean matched FastCDC's on each file (NumPy: 10614 bytes for FastCDC, 10603 for TileCDC at `avg_size`=9856). At matched means the two are equivalent: differences range from -0.31 to +0.38 percentage points. At default parameters, where TileCDC's chunks are smaller, the difference ranges from -0.01 to +1.38.

Two threads. With two threads chunking different files on the VM's two vCPUs, TileCDC's aggregate throughput rose from 4.07 to 8.31 GB/s and the C baseline's from 1.66 to 3.33 GB/s. We do not know whether the two vCPUs share a physical core.

Correctness. Because boundaries must not depend on hardware, the test suite checks the AMX, AVX-512 and scalar implementations against an oracle written directly from the definition, with no blocks or bitmaps: six kinds of data (random, all zeros, a four-letter alphabet, periodic text, word-like text, alternating regions) under four parameter sets, every buffer length within 70 bytes of 0, *min*, *avg* and *max* at three alignments, streaming against whole-buffer chunking,

file	edit	n	FastCDC	TileCDC	matched
NumPy src	insert	10	99.82	99.82	99.79
NumPy src	insert	100	98.19	98.40	98.22
NumPy src	insert	1000	84.28	85.65	84.39
NumPy src	delete	10	99.83	99.82	99.80
NumPy src	delete	100	98.17	98.41	98.24
NumPy src	delete	1000	84.35	85.68	84.43
SciPy bin	insert	10	99.71	99.83	99.82
SciPy bin	insert	100	98.19	98.66	98.56
SciPy bin	insert	1000	88.50	89.04	88.19
SciPy bin	delete	10	99.91	99.90	99.90
SciPy bin	delete	100	98.33	98.69	98.60
SciPy bin	delete	1000	88.78	89.58	88.83

Table 3: Percentage of the edited file covered by previously stored chunks, mean of five seeds. "FastCDC" is the C baseline; "matched" is TileCDC at the average size that matches FastCDC's mean chunk size.

20 random seeds, and all 14 dataset files: 18,078 comparisons, all identical.

6 Discussion and limitations

Hashless chunkers are faster. Chunkers that avoid hashing, such as AE [6], SeqCDC [7] and their vectorized versions in VectorCDC [8, 9], report throughputs from several to tens of GB/s per core, at a cost of a few percent of deduplication on some datasets and failure modes on low-entropy data. TileCDC keeps a hash-based boundary test with Gear-like statistics. It is faster than the two hash-based implementations we measured; we did not run SS-CDC, RapidCDC, SIMD Gear variants or the hashless systems and make no claim relative to them.

Chunking is one stage. A deduplication pipeline also fingerprints every chunk, typically with SHA-256 or BLAKE3 at 1–3 GB/s per core, and often compresses or encrypts. With fingerprinting at 1–3 GB/s, replacing FastCDC by TileCDC at the throughputs above shortens the chunk-and-fingerprint stage by about 23–40%, not by the chunker's speedup; the gain is largest where fingerprints are cheap or run on other cores.

Average size cap. The window hash carries about 17 bits of entropy (§3.3), so TileCDC 0.1 supports average chunk sizes up to 32 KiB. Large-chunk deployments (64 KiB and above, as in some model and dataset stores) need a wider hash, for example a second key tile applied to the same A tiles and combined before the mix, which roughly doubles the tile multiply and store work.

Where it runs. AMX exists only on recent Intel Xeon processors, available in the major clouds but not on laptops, desktops, AMD or most ARM servers. TileCDC therefore targets server-side ingest: backup targets, storage appliances,

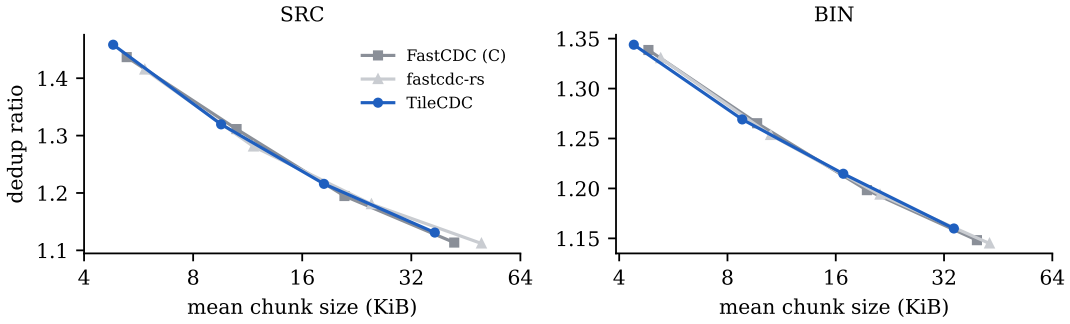


Figure 3: Deduplication ratio against mean chunk size for target averages of 4–32 KiB (min = avg/4, max = 8×avg). Higher is better at a given chunk size. TileCDC tracks the FastCDC curves, -0.21% on average.

and services that rechunk uploaded data. The AVX-512 path covers recent AMD and Intel servers. ARM's Scalable Matrix Extension provides integer outer products on a tile array and might host a similar kernel; we have not tried it. Using AMX also has system costs we did not measure: the first tile instruction in a thread triggers a permission and state allocation, and each thread that uses tiles adds about 8 KB of register state to context switches. All results come from one CPU model in one virtual machine; frequency and power effects of AMX were not studied.

Compatibility. TileCDC's boundaries differ from FastCDC's, so adopting it in an existing repository splits deduplication between old and new data, as any chunker change does. A Gear hash is itself linear over its table values; computing Gear bit-exactly on a matrix unit would avoid this cost but needs 64-bit modular arithmetic that int8 tiles do not offer directly. We leave it open.

Security. Several backup tools key their chunkers to hide content through chunk lengths, and practical attacks recover such keys or content [21, 22]. TileCDC's seed changes its key and substitution, but the hash is linear after substitution and we make no claim that it resists these attacks. Applications that need keyed chunking should add a keyed nonlinear finalization or use a dedicated construction.

7 Related work

CDC. Rabin fingerprints [1] underlie LBFS [2]; Gear [3] and FastCDC [4, 5] made hash-based CDC cheaper with skipping and normalized chunking. Hashless methods replace the hash with byte comparisons: AE [6], SeqCDC [7]. RapidCDC [10] skips ahead using the locality of duplicate chunks. Parallel and SIMD chunkers include SS-CDC [11], which runs independent streams before a sequential pass, and VectorCDC [8, 9], which vectorizes hashless chunkers; Shredder [12] ran Rabin chunking on GPUs. Chonkers [13] gives provable size and locality guarantees. A recent survey compares many of these [14]. None uses a matrix unit, and the hash-based ones keep a rolling recurrence; TileCDC evaluates an independent hash per offset.

Non-GEMM work on matrix units. Convolutions are routinely lowered to matrix multiplication, by materialising windows (im2col) or by computing addresses on the fly (implicit GEMM) [16]. ConvStencil [17] rearranges stencil computations for tensor cores, and Dakkak et al. map reductions and scans onto tensor cores [18]. TileCDC applies the windowed-product idea to integer byte streams, forming the operand with an overlapping row stride of the tile load rather than a layout transformation, and uses the product as a hash rather than a numeric result.

Linear hashing. Hashes defined by Toeplitz matrices are a classic universal family [19, 20]. TileCDC's window hash is a Toeplitz-structured linear map; we use it for its computational shape, not for universality guarantees.

8 Conclusion

A windowed hash evaluated at every byte offset is a matrix product, and an AMX tile load with a stride shorter than its rows forms the overlapping-window operand for free. TileCDC turns this into a chunker that runs 2.2–2.6× faster than FastCDC on one core with deduplication within 1% of FastCDC's, and into an AVX-512 variant for CPUs without AMX, with identical boundaries everywhere. The kernel is not specific to chunking: any fixed linear function over every window of a byte stream, such as similarity features for delta compression [23] or rolling checksums for file synchronisation, can use it. TileCDC is available at `https://github.com/ashutosh-rath02/tilecdc` under the Apache 2.0 licence; `scripts/fetch_data.sh` rebuilds the datasets and `bench/run_all.sh` regenerates every number in this paper.

Acknowledgments. The idea search, implementation, benchmarks and first draft of this paper were produced with the assistance of Claude (Anthropic), an AI system, under the author's direction, and reviewed by a separate AI reviewer.

The author is responsible for the content and for checking every reported result.

References

[1] M. O. Rabin. Fingerprinting by random polynomials. Tech. Rep. TR-15-81, Harvard University, 1981.

[2] A. Muthitacharoen, B. Chen, and D. Mazières. A low-bandwidth network file system. In *SOSP*, 2001.

[3] W. Xia et al. Ddelta: A deduplication-inspired fast delta compression approach. *Performance Evaluation*, 79, 2014.

[4] W. Xia et al. FastCDC: A fast and efficient content-defined chunking approach for data deduplication. In *USENIX ATC*, 2016.

[5] W. Xia et al. The design of fast content-defined chunking for data deduplication based storage systems. *IEEE TPDS*, 31(9), 2020.

[6] Y. Zhang et al. AE: An asymmetric extremum content defined chunking algorithm for fast and bandwidth-efficient data deduplication. In *IEEE INFOCOM*, 2015.

[7] S. Udayashankar et al. SeqCDC: Hashless content-defined chunking for data deduplication. In *ACM Middleware*, 2024.

[8] S. Udayashankar et al. VectorCDC: Accelerating data deduplication with vector instructions. In *USENIX FAST*, 2025.

[9] S. Udayashankar et al. Accelerating data chunking in deduplication systems using vector instructions. arXiv:2508.05797, 2025.

[10] F. Ni and S. Jiang. RapidCDC: Leveraging duplicate locality to accelerate chunking in CDC-based deduplication systems. In *ACM SoCC*, 2019.

[11] F. Ni, X. Lin, and S. Jiang. SS-CDC: A two-stage parallel content-defined chunking for deduplicating backup storage. In *ACM SYSTOR*, 2019.

[12] P. Bhatotia, R. Rodrigues, and A. Verma. Shredder: GPU-accelerated incremental storage and computation. In *USENIX FAST*, 2012.

[13] B. Berger. The Chonkers algorithm: Content-defined chunking with provable strict guarantees on size and locality. arXiv:2509.11121, 2025.

[14] M. Gregoriadis et al. A thorough investigation of content-defined chunking algorithms for data deduplication. arXiv:2409.06066, 2024.

[15] Intel Corporation. *Intel Architecture Instruction Set Extensions and Future Features Programming Reference*. Chapter on Intel AMX.

[16] S. Chetlur et al. cuDNN: Efficient primitives for deep learning. arXiv:1410.0759, 2014.

[17] Y. Chen et al. ConvStencil: Transform stencil computation to matrix multiplication on tensor cores. In *ACM PPoPP*, 2024.

[18] A. Dakkak, C. Li, I. Gelado, J. Xiong, and W.-m. Hwu. Accelerating reduction and scan using tensor core units. In *ACM ICS*, 2019.

[19] Y. Mansour, N. Nisan, and P. Tiwari. The computational complexity of universal hashing. In *ACM STOC*, 1990.

[20] H. Krawczyk. LFSR-based hashing and authentication. In *CRYPTO*, 1994.

[21] K. T. Truong et al. Breaking and fixing content-defined chunking. IACR ePrint 2025/558.

[22] B. Alexeev, C. Percival, and Y. X. Zhang. Chunking attacks on file backup services using content-defined chunking. arXiv:2504.02095, 2025.

[23] Y. Zhang et al. Finesse: Fine-grained feature locality based fast resemblance detection for post-deduplication delta compression. In *USENIX FAST*, 2019.